МАРШРУТ 02

Системный анализ от Junior до Middle и Middle Plus

15 основных блоков · 4 дополнительных блока Middle Plus

Этот маршрут для аналитика, который уже умеет собирать требования, читать SQL и JSON, рисовать последовательности взаимодействий и описывать небольшой API. Здесь нужно научиться самостоятельно вести изменение через несколько компонентов: выяснять неопределённость, выбирать решение вместе с командой, учитывать сбои и выпускать функцию в работающую систему.

Главная разница в ожидаемой работе - самостоятельность и ответственность за согласованность решения. Middle замечает неизвестные условия, сравнивает варианты и доводит задачу до проверяемого результата. Middle Plus дополнительно уверенно ведёт изменения на стыке команд, понимает эксплуатационные последствия и помогает улучшать качество чужой работы. Это рабочие ориентиры программы; единых границ грейдов для всех компаний нет.

Прохождение гида создаёт базу и доказательства навыка. Подтверждать самостоятельность лучше на настоящих задачах или командном учебном проекте с разработчиками и QA. Самостоятельное чтение паттернов не заменяет опыта согласований, выпуска и разбора ошибок.

Содержание

Как устроен второй маршрут 3
Проект развития сервиса записи 4
M00 Проверь входной уровень 5
M01 Управляй неопределённостью и правилами домена 6
M02 Сделай НФТ измеримыми 7
M03 Обсуди архитектуру и границы компонентов 8
M04 Защити данные при одновременных действиях 9
M05 Меняй API без неожиданностей для клиентов 10
M06 Спроектируй обработку событий 11
M07 Определи поведение при сбоях интеграции 12
M08 Измени модель данных и подготовь миграцию 13
M09 Спроектируй доступ и проверь угрозы 14
M10 Сделай поведение системы наблюдаемым 15
M11 Разберись с SOAP gRPC и файловым обменом 16
M12 Проведи фичу через несколько команд 17
M13 Защити полный пакет изменения 18
M14 Покажи самостоятельность на рабочих кейсах 19
P01 Middle Plus Сложная бизнес операция и компенсации 20
P02 Middle Plus Модели чтения кэш и история событий 21
P03 Middle Plus Межкомандная миграция 22
P04 Middle Plus Ревью и развитие практики команды 23
Приложение А Матрица отказов для итогового проекта 24
Приложение Б Шаблоны решений и маппинга 25
Приложение В Вопросы для итоговой самопроверки 26
Приложение Г Практика с ИИ и техническими источниками 27
Реестр источников этого маршрута 28

Как устроен второй маршрут

Сначала пройди диагностику. Уже освоенный блок можно защитить и пропустить. Оценка строится по работе на незнакомой задаче, а не по знакомству с терминами. Затем иди последовательно по M01-M14. Темы Middle Plus отмечены отдельно: они не становятся неожиданным требованием на базовой защите Middle.

Ориентир основной части - около 315 часов. С повторением и доработками планируй 350-420 часов. Это оценка предлагаемой учебной нагрузки, а не данные о сроках чужого курса. При 12 часах в неделю получится около 7-8 месяцев учебной работы; при 20 часах - примерно 4-5 месяцев. В реальном проекте темп зависит от доступности команды и выпуска, поэтому учёт практики и ожидания ведётся отдельно.

Навигация

Блок	Основная работа	Ориентир	Результат
M00	Диагностика	8 ч	Индивидуальный маршрут
M01	Неопределённость и домен	18 ч	Правила, границы и решения
M02	НФТ и оценка нагрузки	22 ч	Измеримые требования
M03	Архитектура и границы	24 ч	C4 и сравнение вариантов
M04	SQL, транзакции, конкуренция	24 ч	Защищённые инварианты
M05	Эволюция API	22 ч	Совместимый контракт
M06	Асинхронные взаимодействия	28 ч	События и гарантии обработки
M07	Надёжность интеграций	24 ч	Матрица сбоев и восстановление
M08	Данные и миграции	20 ч	План изменения данных
M09	Безопасность	20 ч	Права и угрозы
M10	Наблюдаемость и инциденты	20 ч	SLI, метрики и runbook
M11	SOAP, gRPC, файлы	16 ч	Выбор и описание протокола
M12	Ведение фичи и выпуск	24 ч	Согласованная готовность
M13	Итоговый проект и защита	30 ч	Полный пакет изменения
M14	Рабочие кейсы и интервью	14 ч	Портфолио и план роста
P01-P04	Middle Plus по выбору	12-20 ч каждый	Углублённая защита

Сохраняется общая шкала 0-3 по пяти признакам: бизнес-смысл, полнота сценариев, техническая согласованность, проверяемость, ясность объяснения. Для зачёта блока нужно минимум 2 везде и отсутствие критичных замечаний. Итоговую защиту проводи на изменённом условии, чтобы проверить перенос навыка.

Проект развития сервиса записи

Если ты прошёл первый гид, продолжай свой сервис. Если нет - подготовь краткое описание: клиенты записываются на занятия с ограниченными местами, администратор ведёт расписание, уведомления отправляются через внешнюю систему. Клиент может отменять запись до согласованного дедлайна. База хранит клиентов, занятия, записи и историю отправок.

Теперь у сети несколько филиалов, есть мобильный и веб-клиент, появляются интеграции и нагрузка. Задача - добавить временное удержание места перед подтверждением, надёжные уведомления и выгрузку подтверждённых записей во внешнюю учебную платформу. При этом старый клиент должен продолжать работать.

Начальные условия кейса: для новых клиентов удержание действует 5 минут; источник истины о вместимости - наш сервис; уведомление не определяет успех бронирования; внешняя учебная платформа получает уже подтверждённые записи и может отставать; повторная отправка не должна создавать там второго участника. Отмена во внешней платформе отражает нашу отмену, но не подменяет её. Онлайн-оплаты в базовой версии нет.

Что предстоит согласовать: как старый клиент подтверждает запись без нового экрана; что происходит с запросом подтверждения на границе срока; можно ли менять вместимость ниже текущего числа занятых мест; как обрабатывать изменение времени занятия; сколько допускается отставание внешней платформы; что делать при длительном недоступном партнёре. Не выдавай свои решения за требования заказчика.

Одна из допустимых моделей состояний: HELD → CONFIRMED / EXPIRED / CANCELLED; CONFIRMED → CANCELLED. Удержания и подтверждённые записи занимают место; EXPIRED и CANCELLED - нет. Это исходная гипотеза. Выбранные состояния, переходы и правила конкуренции нужно защитить. Внешняя синхронизация имеет отдельное состояние и не должна возвращать запись из CANCELLED в CONFIRMED.

M00 Проверь входной уровень

Результат: ты понимаешь, какие пробелы мешают самостоятельной работе. Ориентир - 8 часов с разбором.

Выполни задания без готового решения, затем защити их. Можно пользоваться документацией, если фиксируешь, что проверял. Ментор не оценивает скорость воспроизведения определения.

1. За 20 минут подготовь вопросы к требованию «добавить перенос занятия». Назови риски, которых нет в исходной фразе.
2. Напиши Use Case отмены с альтернативами и критериями на границе времени.
3. По таблицам clients/sessions/bookings найди занятия без активных записей и число уникальных клиентов за период. Объясни кардинальность JOIN.
4. Опиши POST создания записи: поля, ошибки, источник владельца, повторный запрос.
5. Нарисуй sequence, где backend сохранил результат, но клиент не получил ответ.
6. Измени поле API и перечисли потребителей, которых нужно проверить.

Обязательно:

- S74 Как мы унифицировали техническое интервью аналитика · RU. Сравни формат оценки одной команды с тем, что реально проверили задания. Не делай вывод о грейде только по совпадению вопросов.
- S38 Проектирование спецификации OpenAPI · RU. Используй статью как опору для ревью своего API, а не как готовую шпаргалку.

Правило маршрутизации: если SQL JOIN, сценарии, HTTP или модели данных пока не получаются, вернись к J05, J07-J13 первого гида. Если базовая работа стабильна, переходи к M01. Для разработчика можно сократить знакомые технические блоки после защиты, но не пропускать требования и коммуникацию автоматически. Для бизнес-аналитика, наоборот, часто потребуется больше времени на данные и интеграции.

Сдай: таблицу навыков с доказательством, оценкой 0-3, пробелом и следующим заданием. Запланируй первые две недели, затем пересмотри нагрузку. Не помечай навык как подтверждённый только потому, что прочитал соответствующую статью.

M01 Управляй неопределённостью и правилами домена

Результат: ты превращаешь противоречивые пожелания в набор проверенных решений. Ориентир - 18 часов.

На уровне Middle важно замечать, что ещё неизвестно. «Нужна бронь на пять минут» не отвечает на вопросы об источнике времени, конкурентных действиях, повторе команды и задержке обработки. Выделяй инварианты - условия, которые не должны нарушаться: нельзя занять больше мест, чем есть; один клиент не должен случайно получить две активные записи на занятие.

Разбери предметные термины, события, команды, состояния и границы ответственности. Один термин может означать разные вещи для филиала и платформы. Bounded Context помогает сохранить согласованную модель внутри определённой границы; это не требование превращать каждый термин в микросервис.

Обязательно:

- S47 Bounded Context · EN. Разбери смысл ограниченного контекста и различия моделей. Составь собственный пример «занятия» у нас и у партнёра.
- S05 Работа с требованиями на разных этапах проекта · RU. Прочитай сопровождение требований и работу с изменениями в ходе разработки.
- S03 Gherkin Reference · EN. Используй Scenario Outline для таблицы комбинаций состояний и времени.

Сделай: проведи отдельные интервью с владельцем процесса, поддержкой и разработчиком. Подготовь карту правил удержания, журнал допущений и таблицу решений «условия → действие». Включи одновременную отмену и подтверждение, просроченное удержание, перенос занятия, уменьшение вместимости и повтор клиента. Для каждого спорного вопроса укажи владельца решения и последствия задержки.

Проведи 45-минутный воркшоп: участники раскладывают последовательность бизнес-событий и выявляют расхождения. Зафиксируй принятые правила, а технические гипотезы вынеси отдельно. Называть такую сессию полноценным EventStorming необязательно: важен результат обсуждения и корректное использование выбранной техники.

Самопроверка: какие инварианты действуют всегда? Где источник истины? Какое решение принимает продукт, а какое команда реализации? Что случится, если два контекста по-разному понимают «подтверждено»? Как показать последствия нерешённого вопроса без выдуманного ответа?

Зачёт: ментор намеренно меняет одно условие. Ты находишь затронутые сценарии, состояния и контракты, аргументируешь границу и не теряешь исходную цель.

M02 Сделай НФТ измеримыми

Результат: команда понимает, какое качество требуется и в каких условиях. Ориентир - 22 часа.

Изучи latency, throughput, RPS, конкурентные запросы, процентиль, доступность, допустимую потерю данных и время восстановления. Отличай среднее от p95/p99. SLI - измеритель поведения, SLO - целевой уровень; SLA - отдельная договорённость с последствиями, если она существует. Не называй любой внутренний ориентир SLA.

Нагрузка начинается с допущений: число активных пользователей, действий за период, доля пикового трафика, размер сообщений и срок хранения. Приводи единицы измерения. Расчёт числа серверов без замера производительности конкретной реализации создаёт ложную точность.

Обязательно:

- S64 Service Level Objectives · EN. Прочитай выбор индикаторов и целевых уровней. Выпиши, что именно измеряется и с чьей точки зрения.
- S51 Timeouts retries backoff and jitter · EN. Разбери влияние задержек внешних зависимостей и бюджетов ожидания.
- S65 Signals в OpenTelemetry · EN. Изучи различие метрик, логов и трассировок, чтобы связать требование с измерением.

Сделай: задай учебный профиль 30 000 активных клиентов в день по 12 запросов чтения. Это 360 000 запросов в день, в среднем около 4,2 RPS. Если 20% приходятся на 15 минут открытия записи, пиковая оценка этого окна - около 80 RPS. Покажи, почему среднее по суткам скрывает нагрузку. Для команд создания считай отдельный профиль, а не копируй долю чтения.

Составь 8 НФТ: чтение расписания, подтверждение записи, допустимое отставание внешней платформы, восстановление после сбоя, аудит, доступность и ограничения данных. Для каждого укажи цель, окно измерения, нагрузку, границу системы и способ проверки. Все числа кейса пометь как учебные и передай на согласование. Предложи, что измерить нагрузочным тестом, вместо заявления «нужны три сервера».

Самопроверка: где измерять время ответа? Как учитывать ошибки при расчёте доступности? Что делать с низким объёмом событий для процентилей? Сколько запросов придёт при повторе на трёх уровнях? Чем RTO отличается от RPO? Как ограничение партнёра влияет на наш SLO?

Зачёт: у каждого НФТ есть измерение и проверяемые условия; арифметика воспроизводится; неизвестные параметры названы. Нельзя подменить согласование произвольным красивым числом.

M03 Обсуди архитектуру и границы компонентов

Результат: ты объясняешь устройство системы и сравниваешь варианты без культа технологии. Ориентир - 24 часа.

Разбери модульный монолит, сервисную архитектуру и микросервисы, синхронные и событийные связи, API Gateway, балансировку, кэш и владение данными. Для аналитика важнее последствия границы, чем количество прямоугольников. Если каждый сервис напрямую меняет таблицы соседа, независимость на схеме ещё не означает независимость в работе.

Изучи C4 System Context и Container. В C4 container - исполняемое приложение или хранилище в принятой модели, а не обязательно Docker-контейнер. Глубже иди только там, где диаграмма помогает обсуждению. Отделяй логическую ответственность от физического развёртывания.

Обязательно:

- S45 C4 model · EN. Изучи уровни System Context и Container, абстракции и примеры диаграмм.
- S46 Microservices · EN. Прочитай характеристики микросервисов и их последствия. Сравни с потребностями нашего небольшого сервиса.
- S66 What is a container · EN. Разбери отличие контейнера от виртуальной машины; достаточно понимать роль среды исполнения.

Расширение:

- S48 CQRS · EN. Прочитай CQRS и условия, при которых разделение моделей может оказаться избыточным.
- V10 Банк не купили Digital card сами внедрили · RU. Доклад о сложном межкомандном проекте: выпиши роли, зависимости и решения. Страница подтверждена поиском, воспроизведение не проверено.

Сделай: подготовь контекст и контейнеры текущей версии. Сравни три варианта: оставить уведомления модулем, выделить worker в том же продукте, вынести отдельный сервис. Оцени простоту выпуска, отказоустойчивость, данные, нагрузку и сопровождение. Выбери вариант и напиши ADR: контекст, варианты, решение, последствия, условия пересмотра.

Покажи источник истины по занятиям, удержаниям и синхронизации с партнёром. Подпиши протоколы и направления взаимодействий. Проверь, что выбранная схема объясняет, где именно выполняется защита вместимости и что будет, если worker недоступен.

Самопроверка: зачем здесь отдельный сервис? Кто владеет данными? Что изменится при независимом выпуске? Почему распределённая система сложнее в диагностике? Когда кэш допустим для списка, но опасен как единственная проверка последнего места?

Зачёт: решение соразмерно задаче, альтернативы разобраны, ограничения названы. Ментор может выбрать любой блок на схеме и получить объяснение его ответственности, данных и поведения при отказе.

M04 Защити данные при одновременных действиях

Результат: ты описываешь алгоритм, который сохраняет инвариант при конкуренции. Ориентир - 24 часа.

Изучи ACID, MVCC, уровни изоляции PostgreSQL, блокировки строк, optimistic locking, уникальные ограничения и атомарное условное изменение. Различай бизнес-транзакцию и транзакцию БД. Два HTTP-вызова не становятся одной транзакцией от того, что стоят подряд на sequence.

Нужно увидеть гонку своими глазами. Два клиента прочитали одно свободное место, оба прошли проверку, оба записались. Простая последовательность SELECT → INSERT не гарантирует инвариант без согласованного механизма конкурентного доступа. Конкретный подход выбирается вместе с разработчиком с учётом нагрузки и модели данных.

Обязательно:

- S25 Transaction Isolation · EN. Прочитай Read Committed, Repeatable Read и Serializable, включая необходимость повторов при некоторых конфликтах.
- S23 Ограничения данных в PostgreSQL · RU. Повтори ограничения и объясни, какие инварианты обеспечивает база.
- S27 Window Functions · EN. Освой ROW_NUMBER, PARTITION BY и ORDER BY для диагностических выборок.

Сделай: в двух SQL-сессиях воспроизведи гонку на последнем месте. Затем сравни блокировку строки занятия и атомарное условное изменение счётчика. В варианте со счётчиком увеличение и создание удержания должны коммититься совместно; отмена и истечение обязаны освобождать место без повторного уменьшения. В варианте с подсчётом строк все конкурирующие пути должны соблюдать согласованный протокол блокировки. Уникальность активной записи защищается отдельно.

Подготовь последовательности: два удержания, подтверждение против истечения, две отмены. Для каждой зафиксируй точку принятия решения, допустимый итог и ответ проигравшей стороне. Учти, что фоновая очистка может запаздывать: истёкшее удержание определяется правилом времени, а не только тем, успел ли worker обновить строку.

Добавь SQL-выборки: последнее событие записи через ROW_NUMBER, поиск зависших удержаний и сверка занятых мест. Для запроса отчёта посмотри EXPLAIN на учебной базе и объясни наблюдаемые операции.

Самопроверка: почему CHECK одной строки не решает межстрочную вместимость? Когда UNIQUE недостаточно? Что произойдёт после deadlock или serialization failure? Может ли SELECT FOR UPDATE отсутствующей строки дать ожидаемую защиту? Все ли пути изменения используют одно правило?

Зачёт: инварианты проверены на двух конкурентных сценариях; результат и повтор описаны. Нельзя выдавать выбор уровня изоляции за универсальную защиту без разбора реального алгоритма.

M05 Меняй API без неожиданностей для клиентов

Результат: изменение контракта учитывает действующих потребителей и повторные команды. Ориентир - 22 часа.

Разбери backward compatibility, версионирование, deprecation, optional/required, null/absence, новые enum-значения, пагинацию при изменяемых данных и контрактные проверки. Даже добавление поля может сломать строгого клиента, а новое значение enum - код с закрытым перечнем. Совместимость проверяется на конкретном потребителе, а не объявляется по интуиции.

Опиши идемпотентность команд: область ключа, связь с пользователем и операцией, срок хранения результата, сравнение payload, параллельные повторы и ответ после сбоя. Заголовок с названием Idempotency-Key сам по себе ничего не гарантирует. Контракт должен объяснять поведение, а реализация - обеспечивать его атомарно с эффектом команды либо согласованной моделью восстановления.

Обязательно:

- S39 Как мы ломали и собирали API заново · RU. Разбери последствия изменения API в нескольких командах. Составь свой список потребителей и точек согласования.
- S52 Idempotent requests · EN. Посмотри, как конкретный API описывает идемпотентные запросы. Не переноси срок хранения и детали Stripe как универсальный стандарт.
- S34 OpenAPI 3.1.1 · EN. Проверь совместимость схем, обязательность, enum и описание ошибок в своей OpenAPI 3.1.1.

Расширение:

- S68 Consumer Driven Contracts · EN. Прочитай подход к контрактам, исходящим из ожиданий потребителя.
- S69 Pact Introduction · EN. Узнай, как Pact связывает проверку consumer и provider; писать инфраструктуру тестов аналитику необязательно.

Сделай: добавь удержание к API, сохранив поведение старого клиента через явный адаптационный сценарий. Подготовь таблицу v1/v2: поведение, поля, ошибки, срок поддержки и способ проверки. Опиши 8 повторов: одинаковый ключ/одинаковое тело, одинаковый ключ/другое тело, два одновременных запроса, другой пользователь, потерянный ответ, истёкший ключ, операция в обработке и повтор после неуспешного результата.

Отдельно сравни offset и cursor pagination для расписания, в которое добавляются занятия. Фиксируй стабильный порядок и дополнительный ключ при равных датах. Наличие курсора не гарантирует стабильный снимок без дополнительного решения.

Самопроверка: что является ломающим изменением? Как клиент узнает о deprecation? Можно ли дедуплицировать только по телу запроса? Что произойдёт при сохранении ключа до бизнес-эффекта и сбое между ними? Почему новый enum надо согласовать с мобильным клиентом?

Зачёт: есть diff контракта, матрица потребителей, сценарии повторов и план перехода. Старый клиент проверяется отдельно, а не предполагается совместимым.

M06 Спроектируй обработку событий

Результат: ты описываешь отправку и обработку сообщений, включая дубликаты и изменение порядка. Ориентир - 28 часов.

Изучи событие и команду, producer/consumer, очередь и pub/sub, partition key, consumer group, offset, acknowledgement, повторную доставку, lag, DLQ и replay. Kafka и RabbitMQ решают пересекающиеся задачи, но не являются взаимозаменяемыми коробками с одинаковой моделью поведения.

Разбирай гарантии по границам: запись в БД, публикация, доставка и внешний эффект. At-least-once требует обработки повторов. Обещание exactly-once в одном механизме не означает ровно один внешний HTTP-вызов или одну запись в любой чужой БД. Publisher confirm и consumer acknowledgement тоже подтверждают разные участки пути.

Обязательно:

- V07 Kafka 101 Events · EN. Разбери событие и устройство Kafka на уровне идеи.
- V08 Kafka 101 Partitions · EN. Изучи partitions и роль ключа для выбранного порядка.
- V09 Kafka 101 Consumers · EN. Разбери consumer groups и чтение событий.
- S55 Consumer Acknowledgements and Publisher Confirms · EN. Прочитай отдельно publisher confirms и consumer acknowledgements. Нарисуй границы их гарантий.
- S50 Transactional Outbox · EN. Изучи, какую проблему dual write решает Outbox и почему публикация всё равно может повторяться.

Расширение:

- S56 Apache Kafka Documentation · EN. Выбери документацию своей версии Kafka и проверь детали порядка, групп и доставки.
- S57 AsyncAPI Document · EN. Посмотри описание channel, operation и message в используемой версии AsyncAPI. Не смешивай синтаксис разных версий.

Сделай: спроектируй BookingConfirmed и BookingCancelled для внешней платформы. Опиши eventId, bookingId, версию события, occurredAt, correlationId и данные предметной области. Выбери partition key и объясни, какой порядок сохраняется и где он не обещан. Назначь владельца схемы и правила совместимости.

Подготовь алгоритм публикации через Outbox и обработки через дедупликацию/Inbox или другой обоснованный механизм. Отметка eventId и локальный эффект должны быть согласованы транзакционно. Если consumer вызывает внешний API, одной локальной таблицы недостаточно для атомарности с чужой системой: нужны её идемпотентность и сверка результата.

Опиши сбои до/после коммита, после отправки до отметки Outbox, после локальной обработки до ack. Покажи повтор старого BookingConfirmed после BookingCancelled: бизнес-модель не должна возвращаться назад только из-за поздней доставки. Реши, использовать ли версию агрегата, проверку состояния или другой механизм.

Самопроверка: когда подтверждать обработку? Что доказывает offset? Где порядок существует, а где нет? Когда допустим replay? Почему DLQ не является решением проблемы без владельца и процедуры разбора? Где сломается наивная «ровно одна доставка»?

Контрольная точка 1: проведи 60-минутный разбор контракта и событий с «тимлидом». Проверяется новая вариация: consumer падает после внешнего успешного вызова. Ты показываешь точку неопределённости, восстановление и недостающие гарантии партнёра.

M07 Определи поведение при сбоях интеграции

Результат: для каждой ожидаемой ошибки есть ограниченное и проверяемое поведение. Ориентир - 24 часа.

Разбери connect/read timeout, общий deadline, retry budget, backoff/jitter, circuit breaker, rate limit, дедупликацию и reconciliation. Повторять нужно не все ошибки и не бесконечно. Если повторы включены на нескольких уровнях, нагрузка может многократно вырасти. Таймаут означает неизвестный результат в некоторых сценариях, а не гарантированный отказ.

Reconciliation - сверка локального и внешнего состояния с правилами исправления расхождений. Это отдельный процесс, у которого есть расписание, источник истины, граница автоматического исправления и ответственный за нерешённые случаи. Компенсация в Saga - бизнес-действие с собственными ограничениями, а не магический ROLLBACK всей распределённой операции.

Обязательно:

- S51 Timeouts retries backoff and jitter · EN. Разбери выбор таймаутов и ограниченных повторов. Составь собственную политику для одного внешнего вызова.
- S53 Webhooks · EN. Прочитай повторы, дубликаты и отсутствие универсального порядка callback.
- S49 Saga · EN. Изучи Saga и смысл компенсирующих действий; в базовом кейсе обоснуй, нужна ли она вообще.

Сделай: подготовь матрицу из 12 сбоев синхронизации с платформой: не удалось соединение; ответ не получен; 429; временный 5xx; невосстановимый 4xx; неверная схема; дубликат callback; неизвестный bookingId; отмена пришла раньше подтверждения; партнёр недоступен сутки; результат расходится; ручной повтор после исправления.

Для каждого укажи локальное состояние, доступное пользователю действие, повторимость, ограничения попыток, способ сверки и сигнал поддержке. Бронирование у нас не отменяется из-за отказа уведомления или задержки платформы. Если бизнес захочет другое правило, нужно изменить модель явно, включая последствия для клиента.

Нарисуй sequence для «партнёр создал участника, ответ потерян». Покажи, откуда берётся внешний idempotency key или business reference, как запросить результат и что делать, если партнёр не даёт ни одного механизма. В последнем случае нельзя честно гарантировать отсутствие дубля без изменения взаимодействия.

Самопроверка: какой бюджет ожидания доступен? Как учесть Retry-After? Кто инициирует ручное восстановление? Что делать, если компенсация тоже упала? Как не превратить сверку в перезапись правильного внешнего состояния неправильным локальным?

Зачёт: нет строки «при ошибке повторяем» без условий и завершения. Все неопределённые результаты имеют путь выяснения или явную эскалацию.

M08 Измени модель данных и подготовь миграцию

Результат: данные и работающий код проходят переход без противоречивых версий. Ориентир - 20 часов.

Изучи индексы, составные ключи, селективность, чтение EXPLAIN, хранение истории, soft delete, backfill, expand/contract и совместимость схем. Индекс помогает определённым запросам, но занимает место и влияет на запись. EXPLAIN ANALYZE исполняет запрос: на изменяющих командах и на большой базе это существенное действие, поэтому тренируйся в учебной среде.

Миграция - больше, чем ALTER TABLE. Нужно понять старые данные, код разных версий, заполнение новых полей, проверки качества, время блокировок и возможность отката. Возврат приложения на старую версию не отменяет необратимое изменение данных автоматически.

Обязательно:

- S26 Using EXPLAIN · EN. Прочитай план, оценки, фактические данные и базовые узлы. Не делай вывод о реальном времени только по cost.
- S70 Parallel Change · EN. Разбери expand, migrate и contract как последовательный переход.
- S23 Ограничения данных в PostgreSQL · RU. Проверь ограничения новой модели и последствия их добавления к старым строкам.

Расширение:

- V06 CS50 SQL Week 5 Optimizing · EN. Optimizing: индексы и планы. Примеры SQLite сравнивай с поведением PostgreSQL, а не переноси механически.
- S27 Window Functions · EN. Используй оконные функции для поиска последнего состояния и аномалий истории.

Сделай: добавь held_until и новые состояния без немедленного удаления старых полей. Подготовь план: добавить совместимую схему, выпустить код, заполнить данные, проверить, перевести чтение, дождаться ухода старых потребителей, удалить устаревшее. Для каждого шага укажи условия запуска и остановки.

Напиши контрольные SQL: число строк до/после, NULL в обязательных полях, некорректные переходы, дубликаты, расхождение вместимости. Опиши аудит: кто, когда и почему изменил запись, какой correlationId связывает действия. Учти историю переноса времени занятия: старое событие не должно интерпретироваться по новому расписанию без выбранного правила.

Самопроверка: может ли старый код прочитать новый enum? Что делать со строками без held_until? Когда backfill завершён? Что реально откатывается? Кто принимает решение остановить миграцию? Какие персональные данные избыточны для истории?

Зачёт: план проходит на трёх состояниях среды - до обновления, при смешанных версиях, после завершения. Контрольные запросы проверяют смысл, а не только отсутствие ошибок SQL.

M09 Спроектируй доступ и проверь угрозы

Результат: права и доверенные границы описаны на уровне объектов и операций. Ориентир - 20 часов.

Разбери OAuth 2.0 и OpenID Connect, access/refresh/ID token, scopes, роли и объектные разрешения. OAuth решает делегирование доступа, OIDC добавляет идентификационный слой. JWT - формат токена, он не делает содержимое секретным и не заменяет проверку подписи, issuer, audience и срока действия.

Для браузерного и мобильного клиента изучи современную рекомендацию Authorization Code с PKCE по актуальному security BCP. Не выбирай flow по старой статье без проверки. Различай проверку пользователя, клиента и внешнего webhook. CORS - механизм браузера, а не универсальная защита API от чужих вызовов.

Обязательно:

- S58 OAuth 2.0 · EN. Разбери actors, scopes, access tokens и основные потоки.
- S59 How OpenID Connect Works · EN. Сравни OIDC с OAuth и назначение ID token.
- S60 RFC 9700 OAuth Security Best Current Practice · EN. Точечно прочитай рекомендации по authorization code, PKCE и небезопасным устаревшим вариантам.
- S61 OWASP API Security Top 10 2023 · EN. Выбери угрозы объектного доступа, аутентификации и ограничений ресурсов.
- S21 CORS · RU. Прочитай назначение CORS и предпроверку.

Расширение:

- S62 Authorization Cheat Sheet · EN. Проверь матрицу разрешений и принцип запрета по умолчанию.
- S53 Webhooks · EN. Разбери проверку подписи callback и повторы.

Сделай: нарисуй доверенные границы: клиент, backend, база, worker, внешняя платформа. Опиши получение и использование токенов на уровне контрактов. Составь права на операцию, объект и филиал: администратор одного филиала не должен автоматически видеть всех клиентов сети.

Проведи разбор 8 угроз: чужой bookingId, подмена branchId, административная команда обычным токеном, повтор подписанного callback, просроченный токен, массовое создание удержаний, секрет в логе, избыточные данные в событии. Для каждой укажи контроль и проверку. Нормативные требования конкретной отрасли согласуются со специалистами безопасности; этот блок не заменяет аудит.

Самопроверка: откуда сервер узнаёт владельца? Можно ли ID token отправлять любому API как access token? Как защитить повтор callback? Почему роли недостаточно без проверки филиала? Где нужна маскировка? Что остаётся доступным после отзыва прав?

Зачёт: угрозы связаны с конкретными полями, методами и тестами. Нет утверждения «безопасно, потому что HTTPS/JWT» без остальных проверок.

M10 Сделай поведение системы наблюдаемым

Результат: по сигналам можно понять, что сломалось и какие бизнес-объекты затронуты. Ориентир - 20 часов.

Различай техническую и бизнес-успешность. Уведомление может вернуть HTTP 200, но запись не появится у партнёра из-за ошибки обработки. И наоборот, наш timeout может сопровождаться успешным внешним результатом. Для расследования нужны связность событий, определённые статусы и контроль расхождений.

Изучи structured logs, requestId/correlationId, trace/span, метрики количества, ошибок и задержки, очереди и lag. Не добавляй высококардинальные идентификаторы каждого клиента в обычные метки метрик без оценки стоимости. Полезный лог отвечает на вопрос о действии и результате, не раскрывая токены и избыточные персональные данные.

Обязательно:

- S65 Signals в OpenTelemetry · EN. Прочитай назначение traces, metrics и logs и сравни, какой вопрос решает каждый сигнал.
- S63 Logging Cheat Sheet · EN. Определи события аудита, исключаемые сведения и ограничения доступа к логам.
- S64 Service Level Objectives · EN. Свяжи SLI с пользовательским результатом и целевым уровнем.

Сделай: составь таблицу событий логирования: операция, этап, objectId, correlationId, результат, бизнес-код, разрешённые диагностические поля. Определи метрики: число созданий/подтверждений, доля отказов по причинам, задержка ответа, возраст старейшего Outbox-события, отставание партнёра, количество неразобранных расхождений.

Подготовь три алерта с порогом, окном, смыслом, владельцем и первым действием. Числа - гипотезы до наблюдений, их нужно пересматривать. Напиши runbook «партнёр не получает подтверждённые записи»: какие показатели открыть, как выделить затронутые объекты, какие повторы безопасны и когда эскалировать.

Проведи учебный инцидент: десять минут растёт очередь, callback отстают, пользовательские подтверждения успешны. Объясни влияние на пользователей, критерий остановки новых операций и способ последующей сверки. Решение зависит от согласованного бизнеса, а не только от красного графика.

Самопроверка: что доказывает trace? Чем аудит отличается от диагностического лога? Когда нужна сверка, даже если ошибок нет? Что означает отсутствие метрик? Как избежать алерта без полезного действия? Какие сведения нужны для оценки масштаба?

Зачёт: по runbook другой человек находит проблемный участок и не выполняет небезопасный массовый повтор. Диагностические требования включены в постановку и приёмку.

M11 Разберись с SOAP gRPC и файловым обменом

Результат: ты можешь прочитать незнакомый контракт и выбрать способ взаимодействия по условиям. Ориентир - 16 часов.

Не все системы используют REST/JSON. Для SOAP изучи Envelope/Header/Body/Fault, namespace, роль WSDL и XSD. Для gRPC - описание сервиса и сообщений в proto, типы вызовов, deadline и совместимость полей. Для файла - структуру, кодировку, разделитель, идентификатор пакета, контроль полноты и подтверждение обработки.

Требуемая глубина зависит от работы. Для универсального Middle достаточно читать контракт и формулировать вопросы. Если идёшь в команду с большим SOAP-контуром, практику XSD/WSDL стоит расширить отдельным проектом. Не требуется изучать все протоколы одинаково глубоко.

Обязательно:

- S42 SOAP 1.2 Messaging Framework · EN. Прочитай структуру SOAP и Fault. Найди, где передаются бизнес-данные и где ошибка протокола.
- S43 Введение в gRPC · EN. Разбери proto-контракт и четыре типа RPC.
- S44 Protocol Buffers proto3 · EN. Прочитай правила типов, номера полей и совместимость при изменении сообщений.

Сделай: представь запрос создания участника платформы в трёх вариантах: HTTP/JSON, gRPC и пакетный CSV. Для каждого перечисли подходящие условия, ошибки, задержку, способ корреляции и повтор. Напиши маленький `.proto` с booking_id и external_user_id; отдельно покажи изменение, которое нельзя делать с уже использованным номером поля.

Для CSV опиши заголовки, UTF-8, правила экранирования, часовой пояс, batchId, число строк, контрольную сумму и отчёт построчных ошибок. Определи момент, когда пакет считается принятым и когда обработанным. Покажи неполную загрузку, повтор файла, частичный успех и исправленную повторную отправку. Для передачи персональных данных согласуй канал и разрешения.

Подготовь таблицу маппинга нашей записи в формат партнёра. Включи значения по умолчанию, справочник статусов, неизвестный enum и отсутствие внешнего идентификатора. Вопросы к партнёру должны быть сформулированы так, чтобы ответ влиял на реализацию.

Самопроверка: где искать описание SOAP-операции? Чем gRPC deadline отличается от локального ожидания клиента? Зачем фиксировать номера полей? Как понять, что файл не обрезан? Можно ли повторить весь частично обработанный пакет? Кто хранит результаты построчной обработки?

Зачёт: ты читаешь подготовленный ментором незнакомый пример и называешь структуру, ограничения и неопределённости. Выбор протокола объясняется потребностью, а не тем, что он новее.

M12 Проведи фичу через несколько команд

Результат: требования, контракты, проверки и выпуск согласованы между владельцами. Ориентир - 24 часа.

Изучи декомпозицию вертикального изменения, зависимости, readiness-критерии, release plan, feature flag, обратимую раскатку, согласование контрактов и impact analysis. Для аналитика важно дать командам общий предмет обсуждения и не потерять решения, пока реализация идёт параллельно.

Простой пример: backend уже поддерживает HELD, а старый мобильный клиент знает только CONFIRMED/CANCELLED. Нельзя ограничиться задачей «добавить enum». Нужно решить, увидит ли старый клиент новое состояние, кто преобразует ответ и как проверяется переходный период.

Обязательно:

- S39 Как мы ломали и собирали API заново · RU. Разбери работу с API на стыке команд и изменения уже используемого контракта.
- S70 Parallel Change · EN. Составь переход через смешанные версии и критерии завершения.
- S71 Feature Toggles · EN. Прочитай назначение флагов, их виды и стоимость долгого существования.
- S01 Scrum Guide · EN. Повтори прозрачность результата, review и Definition of Done. Локальные критерии готовности к работе команда определяет сама.

Сделай: разбей удержание мест на поставляемые части. У каждой задачи укажи цель, вход, выход, владельца, зависимость и проверку. Составь матрицу согласований backend/frontend/mobile/QA/поддержка/партнёр. Назначь версию контрактов и дату, после которой изменение проходит impact review.

Подготовь план включения: миграция совместимой схемы, код с выключенной функцией, проверка старого сценария, включение на тестовом сегменте, измерение, расширение, завершение перехода. Для остановки укажи метрики и бизнес-условия. Флаг выключает только то поведение, которое предусмотрено реализацией; он не возвращает уже изменённые данные автоматически.

После учебного груминга собери 5 новых решений из чата в журнал и обнови постановку. В конце проведи 20-минутный обзор для QA: что изменилось, что осталось совместимым, какие риски и что проверять на границах. Открытые вопросы должны иметь владельца, а критичные - блокировать конкретную часть выпуска.

Самопроверка: какая часть может поставляться отдельно? Кто согласует внешний контракт? Где актуальное решение после переписки? Как проверить старый клиент? Чем остановка раскатки отличается от отката? Кто удаляет флаг после завершения?

Контрольная точка 2: проведи учебный refinement перед разработчиком, QA и владельцем продукта. Цель - получить реализуемый план и список решений, а не прочитать документ вслух. Все изменения фиксируются после встречи.

M13 Защити полный пакет изменения

Результат: ты самостоятельно проводишь сложную фичу от потребности до плана эксплуатации. Ориентир - 30 часов.

Итоговая работа - запуск удержаний с надёжными событиями и совместимостью старого клиента. Ментор добавляет новые ограничения после того, как ты согласовал первую версию. Проверяется не объём документов, а их способность выдержать изменение.

Комплект на сдачу:

1. Контекст, проблема, границы, участники, правила и журнал неизвестного.
2. AS IS/TO BE, C4 Context/Container, состояния удержания/записи и внешней синхронизации.
3. Sequence: успешный сценарий, последний слот при конкуренции, потерянный ответ, поздний/повторный callback, отмена во время синхронизации.
4. OpenAPI с переходом старого клиента; событие и его совместимость; интеграционный маппинг.
5. ERD, ограничения, история, ключи идемпотентности, Outbox/дедупликация и правила очистки.
6. НФТ, профиль нагрузки, матрица сбоев, метрики, алерты и runbook.
7. План миграции, выпуска, остановки и сверки; декомпозиция по владельцам.
8. Приёмка, контрактные и конкурентные сценарии; SQL для проверки инвариантов.
9. Два ADR с рассмотренными альтернативами и последствиями.

Материалы для повторения:

- S38 Проектирование спецификации OpenAPI · RU. Проверь контракт и пригодность к реализации.
- S50 Transactional Outbox · EN. Проверь публикацию события вместе с локальным изменением.
- S25 Transaction Isolation · EN. Проверь конкурентные сценарии в выбранной СУБД.
- S64 Service Level Objectives · EN. Проверь измеримость требований.

Формат защиты: 30 минут с заказчиком - цель, правила и компромиссы; 60 минут с тимлидом - данные, API, события, сбои; 45 минут с QA/эксплуатацией - тесты, миграция и диагностика. Дополнительно 90 минут на неизвестное изменение: например, партнёр запрещает повторный reference после отмены или старый клиент не умеет читать новое поле.

По каждому из пяти критериев нужна оценка не ниже 2. Самостоятельность оценивается отдельно: 0 - не движется без решения ментора; 1 - требует пошаговых подсказок; 2 - задаёт нужные вопросы и предлагает работоспособный вариант; 3 - сравнивает варианты и управляет последствиями. Для окончания основного маршрута нужна самостоятельность минимум 2.

Критичные ошибки: превышение вместимости, двойной внешний эффект без пути выяснения, раскрытие чужих данных, возврат отменённой записи в активное состояние из-за позднего события, необратимая потеря данных при миграции без принятого решения. После такого замечания работа возвращается на доработку независимо от общей суммы баллов.

Если есть разработчики и QA, реализуйте минимальный вертикальный сценарий и проверьте его. Если реализации нет, сдавайте аналитический прототип, SQL-демонстрации и тестовые сценарии с явным статусом. Документирование решения не следует выдавать за доказанную промышленную надёжность.

M14 Покажи самостоятельность на рабочих кейсах

Результат: ты можешь объяснить свой вклад и выбрать следующий шаг развития. Ориентир - 14 часов.

Собери три кейса: новая функция, изменение действующей интеграции, проблема/инцидент. Для каждого опиши контекст, личную ответственность, неопределённость, решение, альтернативы, проверку и результат. Если результат не измерялся, так и напиши. «Разработал API» без масштаба и последствий мало что говорит о работе.

Обязательно:

- S74 Как мы унифицировали техническое интервью аналитика · RU. Используй как один пример построения технической оценки; подготовь ответы с опорой на свои работы.
- S75 AI в помощь системному аналитику · RU. Прочитай опыт применения ИИ и оцени, какие шаги можно ускорить, а какие требуют независимой проверки.

Сделай: подготовь рассказ о фиче на 5 и на 15 минут. На диаграмме покажи личную область работы и решения, которые принимал другой участник. Проведи два мока: открытая задача по проектированию и разбор неизвестного дефекта/контракта. Между ними исправь именно выявленные пробелы.

Для тренировки выбери новые домены: выдача заказа в пункте, перенос доставки, запись к врачу без медицинских решений. Начинай с вопросов и ограничений. Затем за 45 минут подготовь границы, модель состояния, интерфейс взаимодействия и один сценарий сбоя. Нельзя автоматически переносить все паттерны своего учебного проекта в каждую задачу.

На работе возьми изменение с несколькими потребителями и пройди его под согласованным ревью. Собери подтверждения: постановка, принятые решения, контрактный diff, результаты приёмки, обратная связь коллег. Уровень подтверждается повторяющейся самостоятельной работой, а не одной удачной защитой.

Самопроверка: какое решение действительно твоё? Какой компромисс пришлось принять? Что оказалось неверным в первой гипотезе? Как ты понял, что выпуск успешен? Что делал, когда участники не соглашались? Какой пробел мешает следующему уровню больше всего?

Зачёт: история согласуется с артефактами, ограничения названы, новая задача решается без заученного сценария. Для Middle Plus выбирай следующие блоки по рабочим потребностям, а не ради полного набора названий.

P01 Middle Plus Сложная бизнес операция и компенсации

Дополнительный блок, 12-20 часов. Возьми расширение: клиент оплачивает место, запись подтверждается после результата платежного партнёра. Это новый домен и новая модель; не подставляй «оплату» в готовую схему без уточнений.

Нужно выяснить авторитетный статус платежа, связь попытки с бизнес-операцией, резервирование места, истечение, отмену, возврат и сверку. Денежное движение, состояние заказа и состояние интеграции не должны сливаться в один флаг SUCCESS. Правила сумм, валюты и округления задаются отдельно. Реальные расчёты, безопасность и отраслевые обязательства согласуются с профильными участниками команды.

Материалы:

- S49 Saga · EN. Разбери оркестрацию, хореографию и смысл компенсации.
- S52 Idempotent requests · EN. Изучи пример контракта повторов внешнего эффекта.
- S53 Webhooks · EN. Разбери поздние уведомления и сверку состояния.

Практика: подготовь модель операции и таблицу «место × платёж × возврат». Разбери успешный платёж после истечения удержания, два callback, отказ возврата и частично завершённую цепочку. Для каждого состояния задай допустимые действия, срок выяснения и ответственного. Сравни вариант без распределённой транзакции с предложением «откатить всё».

Зачёт: ты показываешь, что именно компенсируется, какие действия необратимы и как выясняется неизвестный результат. Название Saga не заменяет алгоритм и не нужно там, где достаточно более простой схемы.

P02 Middle Plus Модели чтения кэш и история событий

Дополнительный блок, 12-20 часов. Сервис расширился, расписание читают намного чаще, чем меняют. Нужно ускорить чтение, сохранив корректную запись на последнее место.

Материалы:

- S48 CQRS · EN. Прочитай CQRS и предупреждение об избыточной сложности.
- S47 Bounded Context · EN. Проверь границы модели и владельца данных.
- S50 Transactional Outbox · EN. Свяжи изменение источника истины с обновлением проекции.

Практика: сравни прямое чтение БД, кэш и отдельную read model. Определи допустимую устарелость списка и место окончательной проверки вместимости. Подготовь правила обновления, удаления, восстановления проекции и контроля расхождений. Рассмотри перенос занятия и отмену: что увидит клиент со старым кэшем?

Отдельно разберись, что CQRS не обязательно требует Event Sourcing, а журнал бизнес-событий сам по себе не является полноценной моделью Event Sourcing. Если предлагаешь хранение событий как источник состояния, нужны версия схемы, порядок, восстановление и правила исправления. Для исходного кейса допустимо аргументированно отказаться от этого решения.

Зачёт: ты называешь выигрыш, стоимость и условия отказа от выбранного подхода. Устаревшее чтение не нарушает инвариант записи.

P03 Middle Plus Межкомандная миграция

Дополнительный блок, 12-20 часов. Нужно заменить внешнюю учебную платформу, пока часть филиалов ещё работает со старой.

Материалы:

- S70 Parallel Change · EN. Используй поэтапный переход со смешанными версиями.
- S68 Consumer Driven Contracts · EN. Выдели реальные ожидания каждого потребителя.
- S71 Feature Toggles · EN. Разбери управление включением и завершение жизни флагов.

Практика: опиши маршрутизацию по филиалу, идентификаторы старой/новой систем, перенос истории, сверку и прекращение записи в старую платформу. Для каждой операции выбери одного владельца результата. Если предлагаешь двойную запись, покажи, как обрабатываются расхождения и почему это нужно; не считай её автоматически безопасной.

Подготовь план пилота, критерии расширения, условия остановки и список данных, которые нельзя просто откатить. Проведи change review с ролями двух команд, поддержки и QA. После встречи внеси изменения в актуальные контракты.

Зачёт: переходное состояние описано так же подробно, как целевое. У каждой несогласованности есть владелец и способ обнаружения.

P04 Middle Plus Ревью и развитие практики команды

Дополнительный блок, 12-20 часов. Возьми чужую постановку или специально подготовленный документ с ошибками. Задача - повысить качество решения и помочь автору, сохранив ясность ответственности.

Материалы:

- S74 Как мы унифицировали техническое интервью аналитика · RU. Посмотри, как команда унифицирует оценку; перенеси идею общих критериев на ревью.
- S08 Как сделать BPMN диаграмму чуточку лучше · RU. Используй конкретные ошибки нотации как пример проверяемых замечаний.
- S75 AI в помощь системному аналитику · RU. Сравни ручное ревью с помощью ИИ и отдельно проверь ложные замечания модели.

Практика: раздели замечания на критичные, существенные и редакционные. Для каждого укажи фрагмент, риск и вопрос или предложение. Создай маленький чек-лист из повторяющихся дефектов. Проведи разбор с автором и проверь обновлённую работу. Зафиксируй, что улучшилось и какие правила нужно добавить в шаблон.

Измеряй время до содержательной обратной связи, долю повторных критичных ошибок и причины возвратов. Количество комментариев и страниц не равно качеству. Один общий шаблон не должен заставлять каждую маленькую фичу производить лишние документы.

Зачёт: автор понимает причину замечаний и самостоятельно исправляет решение. Ты можешь отличить свой стилевой вкус от ошибки и объяснить, когда достаточно локальной договорённости.

Приложение А Матрица отказов для итогового проекта

Это минимальные строки для собственной матрицы. Политики и числа должны следовать из принятых решений, а не копироваться из таблицы автоматически.

Ситуация	Что нужно определить	Чем проверить
Два удержания последнего места	Единственную точку конкурентного решения и ответ проигравшему	Две параллельные команды, проверка вместимости
Ответ клиенту потерян после коммита	Получение результата и повтор команды	Повтор с тем же ключом без второго эффекта
Подтверждение на границе held_until	Правило времени и допустимый переход	Контролируемое время и конкурентный запуск
Worker истечения отстал	Семантическое истечение независимо от задержки очистки	Просроченная строка при живом API
Outbox отправлен без отметки	Повторная публикация без повторного бизнес-эффекта	Сбой между отправкой и отметкой
Consumer обработал до падения	Повтор и согласованность дедупликации	Падение до ack, повторная доставка
Партнёр выполнил запрос без ответа	Внешний ключ, запрос статуса и сверка	Искусственно потерянный ответ
Позднее подтверждение после отмены	Защиту состояния и обработку старого события	Переставленные события
Старый клиент увидел новый enum	Переходный контракт или адаптацию	Контрактная проверка старой версии
Миграция заполнена частично	Совместимость чтения и критерии продолжения	Смешанный набор старых и новых строк
Сверка нашла расхождение	Источник истины и границу автоматического исправления	Подготовленные пары разных состояний
Длительный сбой партнёра	Допустимое отставание и реакцию поддержки	Рост возраста очереди и учебный runbook

Приложение Б Шаблоны решений и маппинга

ADR архитектурное решение

Название и статус. Какое решение рассматривается, кто согласует, принято ли оно. **Контекст.** Проблема, инварианты, нагрузка и ограничения. **Варианты.** Не меньше двух осуществимых решений. **Выбор.** Что делаем и почему. **Последствия.** Дополнительная сложность, риски, сопровождение. **Проверка.** Что покажет работоспособность. **Пересмотр.** При каком изменении допущений вернёмся к выбору.

Пример темы: «Публикация событий подтверждения через Outbox». Альтернатива - отдельная отправка после коммита с механизмом восстановления. Сравнение должно учитывать потерю события между записью и публикацией, повторы и стоимость сопровождения. Не выдавай Outbox за единственный допустимый ответ.

Таблица маппинга

Поле партнёра	Источник	Правило	Ошибка или отсутствие
externalReference	booking.id	Стабильный идентификатор нашей записи	Без него запрос не отправляется
personId	clients.external_platform_id	Используется согласованный внешний идентификатор	Отдельная процедура связывания, без выдуманного значения
status	booking.status	Только опубликованные бизнес-состояния по справочнику	Неизвестное состояние не превращаем молча в CONFIRMED
occurredAt	Время доменного события	Однозначное время с согласованным форматом	Некорректное значение попадает в разбор
branchCode	Справочник соответствий филиалов	Версионируемое соответствие	Ошибка справочника блокирует конкретную запись

Для реального контракта добавь типы, длины, обязательность, примеры и версию справочника. Если правило вычисления зависит от бизнес-события, укажи его явно: текущее состояние строки и историческое значение могут отличаться.

Приложение В Вопросы для итоговой самопроверки

Вопрос	Ориентир хорошего ответа
Как доказать уровень Middle	Самостоятельные решения на новых условиях, согласованные артефакты, доведение до проверки
Почему мало happy path	Сбои и конкурентные действия меняют данные и пользовательский результат
Когда нужен отдельный сервис	Есть обоснованная граница ответственности/выпуска/нагрузки, стоимость распределения учтена
Что защищает бизнес-инвариант	Согласованный алгоритм и ограничения на всех путях изменения
Почему retry опасен	Может повторить эффект, усилить нагрузку и скрыть неизвестный результат
Что решает Outbox	Согласованность локального изменения и записи намерения публикации; дубликаты остаются возможны
Почему Inbox недостаточно для внешнего HTTP	Локальная транзакция не атомарна с внешним эффектом
Чем компенсация отличается от rollback	Это отдельное бизнес-действие с возможными ошибками и необратимыми последствиями
Что проверять при новом enum	Все потребители, десериализацию, ветвления и поведение смешанных версий
Почему SLO требует границы	Пользовательское и внутреннее измерение могут описывать разные результаты
Как сделать миграцию управляемой	Переходные версии, проверки данных, условия остановки и реальный план восстановления
Как обрабатывать неизвестный статус партнёра	Не угадывать смысл; изолировать случай, наблюдать и согласовать правило
Что даёт correlationId	Связь действий; не гарантирует полноту логов и не заменяет идентификатор бизнес-операции
Почему CORS не авторизация	Это браузерный механизм доступа к ответу; прямые запросы требуют серверных проверок
Как ревью помогает автору	Конкретный риск, фрагмент и объяснение, по которому можно самостоятельно исправить работу

Приложение Г Практика с ИИ и техническими источниками

Используй ИИ для поиска противоречий, генерации дополнительных комбинаций и репетиции вопросов. Передавай обезличенные учебные данные. Для каждой существенной технической рекомендации требуй ссылку на документацию и проверяй её сам. Модель может перепутать гарантию Kafka с гарантией внешней операции или синтаксис разных версий OpenAPI.

Пример запроса: «Проверь этот алгоритм подтверждения места при двух конкурентных запросах. Для каждого предполагаемого сбоя покажи шаги A1/B1/A2/B2. Не считай SELECT и INSERT атомарными. Отдельно отметь, какие сведения о транзакции и ограничениях отсутствуют».

Ещё один: «Ты consumer старой версии API. Найди изменения, которые могут нарушить мой разбор ответа. Раздели доказанные несовместимости и вопросы к конкретной библиотеке. Не называй новое поле безопасным без оговорённых ожиданий потребителя».

Финальная проверка - собственная работа: SQL-эксперимент, разбор контракта, тестовый сценарий или ревью с разработчиком. Список паттернов и сгенерированный документ не заменяют этого результата.

Реестр источников этого маршрута

Проверка ссылок выполнена 16 октября 2026 года. Источники подобраны под действия в блоках; справочники не нужно читать целиком. У видеоматериалов подтверждена страница или индексация, но не полное воспроизведение. Нормативные документы и продуктовые примеры используются с указанной версией и контекстом. Полный реестр с доступностью входит в комплект.

Код	Материал и ссылка	Язык	Проверка
S01	Scrum Guide	EN	Открыта
S03	Gherkin Reference	EN	Открыта
S05	Работа с требованиями на разных этапах проекта	RU	Открыта
S08	Как сделать BPMN диаграмму чуточку лучше	RU	Открыта
S21	CORS	RU	Открыта
S23	Ограничения данных в PostgreSQL	RU	Открыта
S25	Transaction Isolation	EN	Открыта
S26	Using EXPLAIN	EN	Открыта
S27	Window Functions	EN	Открыта
S34	OpenAPI 3.1.1	EN	Открыта
S38	Проектирование спецификации OpenAPI	RU	Открыта
S39	Как мы ломали и собирали API заново	RU	Открыта
S42	SOAP 1.2 Messaging Framework	EN	Открыта
S43	Введение в gRPC	EN	Открыта
S44	Protocol Buffers proto3	EN	Открыта
S45	C4 model	EN	Открыта
S46	Microservices	EN	Открыта
S47	Bounded Context	EN	Открыта
S48	CQRS	EN	Открыта
S49	Saga	EN	Открыта
S50	Transactional Outbox	EN	Открыта
S51	Timeouts retries backoff and jitter	EN	Открыта с перенаправлением
S52	Idempotent requests	EN	Открыта
S53	Webhooks	EN	Открыта
S55	Consumer Acknowledgements and Publisher Confirms	EN	Открыта
S56	Apache Kafka Documentation	EN	Страница выбора документации открыта
S57	AsyncAPI Document	EN	Открыта

Код	Материал и ссылка	Язык	Проверка
S58	OAuth 2.0	EN	Открыта
S59	How OpenID Connect Works	EN	Открыта
S60	RFC 9700 OAuth Security Best Current Practice	EN	Открыта
S61	OWASP API Security Top 10 2023	EN	Открыта с перенаправлением
S62	Authorization Cheat Sheet	EN	Открыта
S63	Logging Cheat Sheet	EN	Открыта
S64	Service Level Objectives	EN	Открыта
S65	Signals в OpenTelemetry	EN	Открыта
S66	What is a container	EN	Открыта
S68	Consumer Driven Contracts	EN	Открыта
S69	Pact Introduction	EN	Открыта
S70	Parallel Change	EN	Открыта
S71	Feature Toggles	EN	Открыта
S74	Как мы унифицировали техническое интервью аналитика	RU	Открыта
S75	AI в помощь системному аналитику	RU	Открыта
V06	CS50 SQL Week 5 Optimizing	EN	Страница лекции открыта Плеер не проверен
V07	Kafka 101 Events	EN	Страница урока открыта Плеер не проверен
V08	Kafka 101 Partitions	EN	Страница урока открыта Плеер не проверен
V09	Kafka 101 Consumers	EN	Страница урока открыта Плеер не проверен
V10	Банк не купили Digital card сами внедрили	RU	Подтвержден поиском Плеер не проверен